

CS2006 – Operating Systems

Chapter 10: Virtual Memory

Complete Exam Preparation Notes

Demand Paging · Page Fault · EAT · FIFO · OPT · LRU · Second Chance · Frame Allocation · Buddy System

Instructor: Anaum Hamid | Spring 2026

Table of Contents

1. What is Virtual Memory?
2. Background & Benefits
3. Demand Paging – Concepts & Diagram
4. Valid-Invalid Bit & Page Fault Handling
5. Performance of Demand Paging & EAT
6. Copy-on-Write (COW)
7. Page Replacement – Overview
8. FIFO Page Replacement (Solved Trace)
9. Optimal (OPT) Page Replacement (Solved Trace)
10. LRU Page Replacement (Solved Trace)
11. LRU Approximation: Second Chance / Clock Algorithm (Solved Trace)
12. Frame Allocation – Concepts & Numericals
13. Thrashing
14. Buddy System (Kernel Memory Allocation)
15. Slab Allocator
16. Other Considerations
17. Practice Questions – Theory
18. Practice Questions – Numericals
19. Quick Reference Summary

1. What is Virtual Memory?

Virtual memory is an OS memory-management technique that uses both hardware and software to allow a computer to compensate for physical memory shortages by temporarily transferring data from RAM to disk storage.

- Virtual memory is NOT the physical RAM on memory chips.
- It is an area on the hard drive (swap space) the OS uses as overflow for physical RAM.
- When process demands exceed available RAM, the OS copies rarely-used blocks (pages) to disk and releases that RAM for the current process.
- When a swapped block is needed again, the OS brings it back in, moving another block out if necessary.

□ Key insight: only the PART of a program currently being used needs to be in physical memory. The rest can live on disk until needed.

2. Background & Benefits of Virtual Memory

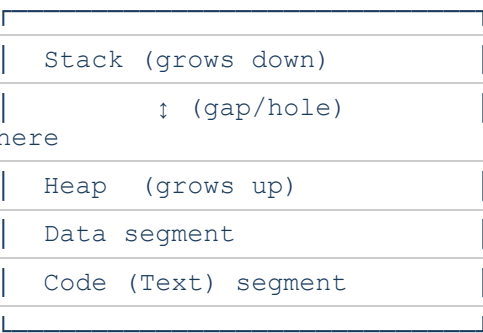
Virtual memory achieves separation of the user's logical memory from physical memory, providing several key advantages:

Benefit	Explanation
Larger logical address space	Program's address space can be much larger than physical RAM
Address space sharing	Multiple processes can share the same address space regions
Efficient process creation	fork() / exec() create processes faster using COW
Higher multiprogramming	More processes run concurrently in the same physical RAM
Less I/O	Only needed pages are loaded; no full program load required

Implementation Methods

- Demand Paging — load pages only when they are accessed (most common)
- Demand Segmentation — load segments only when accessed (less common)

Virtual vs Physical Address Space Diagram

Virtual Memory Address Space Layout	
 A diagram showing the layout of virtual memory address space. It consists of a large rectangle divided into several horizontal sections. From top to bottom: 1. A section labeled 'Stack (grows down)' with an arrow pointing left to '← high addresses'. 2. A section labeled '↑ (gap/hole) here' with an arrow pointing left to '← sparse - no physical frames allocated'. 3. A section labeled 'Heap (grows up)'. 4. A section labeled 'Data segment'. 5. A section labeled 'Code (Text) segment' with an arrow pointing left to '← low addresses'. The entire diagram is enclosed in a box labeled 'Virtual Address Space (large)'.	
Only pages that are ACCESSED get physical frames. Rest stays on disk.	

3. Demand Paging – Concepts & Diagram

Demand paging is the dominant virtual memory technique. Pages are brought into physical memory **ONLY** when they are needed (demanded), not pre-loaded.

Concept	Explanation
Lazy Swapper / Pager	Never swaps a page in unless it will be needed; the swapper that works with pages is called a pager
Less I/O	Only needed pages are transferred — faster startup
Less Memory	Unused pages consume no physical frames
Faster Response	Process starts quicker (no full load)
More Users	More processes can be active simultaneously

Page Access Decision

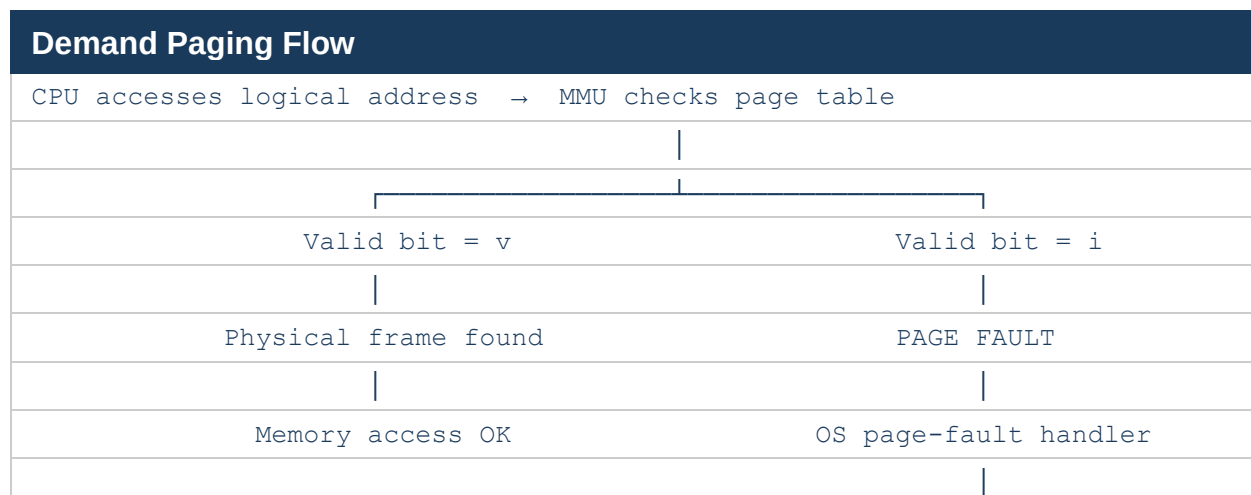
When a page is referenced:

1. Check valid-invalid bit in the page table entry.
2. Valid (v) → page is in memory → continue normally.
3. Invalid (i) → page is not in memory → PAGE FAULT → OS must handle it.

Demand Paging Hardware Requirements

- Page table with valid/invalid bit per entry.
- Secondary storage (swap device / swap space) — fast disk for swapped pages.
- Instruction restart capability — CPU must be able to restart the EXACT instruction that caused the fault after the page is brought in.

Demand Paging Diagram



memory	Find free frame in physical
	Swap page in from disk
	Update page table (set bit = v)
	Restart the faulting instruction

□ Exam Tip: Pure demand paging = process starts with NO pages in memory. First instruction immediately faults. Every access faults until pages are loaded. In practice, locality of reference means a working set loads quickly.

4. Valid-Invalid Bit & Page Fault Handling

Valid-Invalid Bit

Each page table entry carries one bit:

Bit Value	Meaning
v (valid)	Page IS in physical memory — access is legal and fast
i (invalid)	Page is NOT in memory — triggers a page fault trap to OS

Initially ALL entries are marked invalid (i). The bit is set to valid (v) when the OS brings the page into a frame.

Page Fault Steps (from lecture)

4. CPU references a page → MMU finds valid-invalid bit = i → TRAP to OS (page fault).
5. OS looks at an internal table to decide: Is this an invalid reference? → ABORT the process. Or: page just not in memory → continue.
6. OS finds a FREE FRAME in physical memory.
7. OS schedules a DISK OPERATION to swap the page into the frame.
8. OS RESETS the page table entry: frame number set, valid bit set to v.
9. OS RESTARTS the instruction that caused the fault — process continues normally.

Page Fault Handling Diagram
1. Process executing → references page P
2. Page table: P has bit = i → TRAP to OS
↓
3. OS checks: valid logical address? YES → proceed
↓
4. Find free frame (e.g., Frame 14)
↓
5. Read page P from swap disk into Frame 14 ← disk I/O (slow!)
↓
6. Update page table: P → Frame 14, bit = v
↓
7. Restart the faulting instruction
↓
8. Instruction completes normally

□ If there is no free frame, the OS must run a PAGE REPLACEMENT ALGORITHM to select a victim page to evict. This may require TWO disk I/O operations (write victim out + read new page in), doubling the page fault cost.

5. Performance of Demand Paging & Effective Access Time (EAT)

Page Fault Rate

Let p = page fault rate ($0 \leq p \leq 1$)

- $p = 0 \rightarrow$ no page faults $\rightarrow$ EAT = memory access time (ideal)
- $p = 1 \rightarrow$ every reference causes a fault (worst case)

EAT Formula for Virtual Memory

$$\text{EAT} = (1 - p) \times ma + p \times (\text{page fault time})$$

ma = memory access time | page fault time = fault overhead + swap out + swap in

Page fault time components:

10. Service the page-fault interrupt ($\sim 1\text{--}100\ \mu\text{s}$)
11. Read in the page from disk (swap in) — most expensive ($\sim 8\ \text{ms}$ typical)
12. Restart the process

□ If a dirty page must be evicted first, there are TWO disk transfers (swap out + swap in), roughly DOUBLING page fault time. The formula can then include both transfers.

Solved EAT Example (from lecture)

Given:

- Memory access time (ma) = 200 ns
- Average page-fault service time = 8 ms = 8,000,000 ns
- Page fault rate $p = 1 / 1,000 = 0.001$

```
EAT = (1 - p) × ma + p × page_fault_time
EAT = (1 - 0.001) × 200 + 0.001 × 8,000,000
EAT = 0.999 × 200 + 0.001 × 8,000,000
EAT = 199.8 + 8,000
EAT = 8,199.8 ns ≈ 8.2 microseconds
```

□ Even ONE page fault per 1,000 accesses causes a 41× slowdown (8,200 ns vs 200 ns). This shows why minimizing page faults is critical.

EAT – Further Examples

Example 2: Given slowdown target

If $ma = 200\ \text{ns}$, page fault time = 8 ms, and we want $\text{EAT} < 220\ \text{ns}$ ($< 10\%$ degradation):

```
220 > (1-p) × 200 + p × 8,000,000
220 > 200 - 200p + 8,000,000p
220 - 200 > 7,999,800p
20 / 7,999,800 > p
p < 0.0000025 → fewer than 1 fault per 400,000 accesses!
```

Example 3: With swap-out cost

ma = 100 ns, page fault service = 20 ms, dirty-page probability = 70%, clean page fault time = 15 ms

```
Weighted page fault time = 0.70 × 20ms + 0.30 × 15ms = 14 + 4.5 = 18.5 ms =
18,500,000 ns
EAT (p=0.001) = 0.999 × 100 + 0.001 × 18,500,000
               = 99.9 + 18,500 = 18,599.9 ns ≈ 18.6 μs
```

□ Exam Tip: Unit conversions: 1 ms = 10^{-3} s = 1,000,000 ns = 1,000 μs. Always convert everything to the SAME unit before computing EAT.

6. Copy-on-Write (COW)

COW is an optimization for the `fork()` system call. Instead of copying all parent pages to the child, both initially share the same physical frames.

How It Works

13. Parent calls `fork()` → child is created.
14. Parent's pages are marked copy-on-write (shared, read-only).
15. If either parent OR child READS a shared page → no copy needed, direct access.
16. If either parent OR child WRITES to a shared page → OS creates a private copy for the writer.

Without COW	With COW
All parent pages copied at <code>fork()</code>	Only MODIFIED pages copied
Expensive if child calls <code>exec()</code> immediately	Very fast <code>fork()</code> — no wasted copies
Memory: parent size × 2	Memory: only diverged pages duplicated

□ `vfork()` is an extreme variant: parent is suspended and child uses parent's address space directly (no copy at all). Child must call `exec()` immediately.

7. Page Replacement – Overview

When a page fault occurs and there is NO free frame, the OS must select a victim page to evict from memory.

Basic Page Replacement Steps

17. Find the location of the desired page on disk.
18. Find a free frame:
 - If free frame available → use it directly.
 - If no free frame → run replacement algorithm → select victim frame.
 - If victim is dirty (modified) → write victim to disk first (adds disk I/O).
19. Bring desired page into the freed frame; update page table.
20. Restart the process.

Evaluation Criterion

Algorithms are evaluated by running them on a reference string (sequence of page numbers accessed) and counting page faults. Fewer faults = better algorithm.

Algorithm	Strategy	Property
FIFO	Replace oldest loaded page	Simple; suffers Belady's anomaly
OPT (MIN)	Replace page used furthest in future	Optimal (minimum faults); not implementable
LRU	Replace least recently used page	Near-optimal; needs hardware support
Second Chance	FIFO + reference bit	LRU approximation; practical

Belady's Anomaly

With FIFO, adding MORE frames can sometimes cause MORE page faults. This counter-intuitive behaviour is called Belady's Anomaly. OPT and LRU do NOT suffer from it.

Reference Strings Used in This Chapter

FIFO / Second Chance: 6, 0, 1, 2, 0, 3, 0, 5, 2, 3, 0, 3, 0, 3, 2, 1, 2, 0, 1, 6, 0, 1 (22 references, 3 frames)

OPT / LRU: 3, 0, 1, 2, 7, 0, 3, 0, 4, 2, 3, 0, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 (23 references, 3 frames)

8. FIFO Page Replacement – Fully Solved Trace

Reference string: 6, 0, 1, 2, 0, 3, 0, 5, 2, 3, 0, 3, 0, 3, 2, 1, 2, 0, 1, 6, 0, 1 | Frames = 3

Algorithm Rule

Replace the page that has been in memory the LONGEST (arrived first). Track arrival order using a FIFO queue. The front of the queue = oldest page = victim on a miss.

FIFO Trace (3 frames) — F = Page Fault, Red = Replaced Page

Frame/Ref	6	0	1	2	0	3	0	5	2	3	0	3	0	3	2	1	2	0	1	6	0	1
Frame 1	6	6	6	2	2	2	2	5	5	5	0	0	0	0	0	0	0	0	6	6	6	
Frame 2		0	0	0	0	3	3	3	2	2	2	2	2	2	2	1	1	1	1	0	0	
Frame 3			1	1	1	1	0	0	0	3	3	3	3	3	3	2	2	2	2	2	1	
Fault?	F	F	F	F		F	F	F	F	F	F					F	F			F	F	F

Total Page Faults: 15 Hits: 7 Hit Ratio: 31.8%

Step-by-Step Commentary

6: Empty frames → MISS. Load 6. Queue=[6]
0: 0 not in {6} → MISS. Load 0. Queue=[6,0]
1: 1 not in {6,0} → MISS. Load 1. Queue=[6,0,1]
2: MISS. Evict 6 (oldest). Load 2. Queue=[0,1,2]
0: 0 in frames → HIT
3: MISS. Evict 0 (oldest). Load 3. Queue=[1,2,3]
0: MISS. Evict 1 (oldest). Load 0. Queue=[2,3,0]
5: MISS. Evict 2 (oldest). Load 5. Queue=[3,0,5]
2: MISS. Evict 3 (oldest). Load 2. Queue=[0,5,2]
3: MISS. Evict 0 (oldest). Load 3. Queue=[5,2,3]
0: MISS. Evict 5 (oldest). Load 0. Queue=[2,3,0]
3: 3 in frames → HIT
0: 0 in frames → HIT
3: 3 in frames → HIT
2: 2 in frames → HIT
1: MISS. Evict 2 (oldest). Load 1. Queue=[3,0,1]
2: MISS. Evict 3 (oldest). Load 2. Queue=[0,1,2]
0: 0 in frames → HIT
1: 1 in frames → HIT
6: MISS. Evict 0 (oldest). Load 6. Queue=[1,2,6]
0: MISS. Evict 1 (oldest). Load 0. Queue=[2,6,0]
1: MISS. Evict 2 (oldest). Load 1. Queue=[6,0,1]

□ Exam Tip: FIFO result: 15 page faults out of 22 references. FIFO is simple but NOT optimal. Remember: FIFO can suffer Belady's anomaly.

9. Optimal (OPT / MIN) Page Replacement – Fully Solved Trace

Reference string: 3, 0, 1, 2, 7, 0, 3, 0, 4, 2, 3, 0, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 | Frames = 3

Algorithm Rule

Replace the page that will NOT be used for the LONGEST period in the future. Requires knowledge of the future reference string — used as a benchmark only, not implementable in real systems.

OPT Trace (3 frames) — F = Page Fault, Red = Victim (furthest future use)

Frame/Ref	3	0	1	2	7	0	3	0	4	2	3	0	3	0	3	2	1	2	0	1	7	0	1
Frame 1	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
Frame 2		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Frame 3			1	2	7	7	7	7	4	2	2	2	2	2	2	2	2	2	2	2	7	7	7
Fault?	F	F	F	F	F				F	F							F				F		

Total Page Faults: 9 Hits: 14 Hit Ratio: 60.9%

Key Decision Points Explained

Step (ref)	Frames before	OPT decision
4→2: ref=2	{3,0,2}	Replace? Next uses: 3→pos7, 0→pos6, 2→pos10. Replace 2 (furthest). Load 1
4→5: ref=7	{3,0,1}	Next uses: 3→pos7, 0→pos6, 1→pos17. Replace 1 (used latest). Load 7
9: ref=4	{3,0,7}	Next uses: 3→pos11, 0→pos12, 7→pos21. Replace 7 (used latest). Load 4
10: ref=2	{3,0,4}	4 never used again → replace 4. Load 2
17: ref=1	{3,0,2}	3 never used again after here → replace 3. Load 1
21: ref=7	{1,0,2}	2 never used again → replace 2. Load 7

□ OPT result: 9 page faults out of 23 references. OPT gives the MINIMUM possible page faults for this reference string. Use it as a benchmark to evaluate other algorithms.

10. LRU (Least Recently Used) – Fully Solved Trace

Reference string: 3, 0, 1, 2, 7, 0, 3, 0, 4, 2, 3, 0, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 | Frames = 3

Algorithm Rule

Replace the page that was used LEAST RECENTLY (has the oldest last-use timestamp). LRU looks BACKWARD in time — it is OPT looking backward. Does NOT suffer from Belady's anomaly.

LRU Trace (3 frames) — F = Page Fault

Frame/Ref	3	0	1	2	7	0	3	0	4	2	3	0	3	0	3	2	1	2	0	1	7	0	1
Frame 1	3	3	3	2	2	2	3	3	3	2	2	2	2	2	2	2	2	2	2	2	7	7	7
Frame 2		0	0	0	7	7	7	7	4	4	4	0	0	0	0	0	1	1	1	1	1	1	1
Frame 3			1	1	1	0	0	0	0	0	3	3	3	3	3	3	3	3	0	0	0	0	0
Fault?	F	F	F	F	F	F	F		F	F	F	F					F		F		F		

Total Page Faults: 14 Hits: 9 Hit Ratio: 39.1%

LRU Implementation Methods

Counter Implementation	Stack Implementation
Each page table entry has a clock counter	Maintain a doubly-linked stack of page numbers
On each reference: copy clock value to counter	On reference: move page to TOP of stack
Victim: page with smallest counter value	Victim: page at BOTTOM of stack
Requires search through all entries	No search needed — bottom is always LRU victim
Hardware clock needed	6 pointer changes per access (more expensive per access)

□ Exam Tip: LRU result: 14 page faults. LRU typically performs close to OPT. It needs hardware support (counter or stack), making it expensive. That's why LRU approximation algorithms exist.

11. LRU Approximation – Second Chance (Clock) Algorithm

LRU Approximation Algorithms Overview

LRU needs special hardware and is still slow. Three approximation algorithms exist:

Algorithm	Key Idea
Reference Bit Algorithm	One bit per page; set to 1 on access; replace any page with bit=0
Second Chance Algorithm	FIFO + reference bit; give pages a 'second chance' before evicting
Enhanced Second Chance	Uses both reference bit AND modify (dirty) bit for smarter decisions

Second Chance (Clock) Algorithm – Detail

This is essentially FIFO with a hardware-provided reference bit. A clock hand sweeps through frames in FIFO order:

- 21. When a page is REFERENCED (accessed), set its reference bit = 1.
- 22. On a page fault, scan from clock hand position:
 - **Ref bit = 0:** Replace this page (evict it). Load new page here. Advance hand.
 - **Ref bit = 1:** Give page a second chance: set bit = 0, advance hand, check next page.
- 23. If all pages had bit=1 (all get a second chance), eventually the hand comes back and evicts the first one encountered (now with bit=0).

Clock Algorithm Diagram
Clock Hand → Frame 0 [Page A, ref=0] ← Clock hand starts here Frame 1 [Page B, ref=1] ← ref=1: give second chance Frame 2 [Page C, ref=0] ← VICTIM (ref=0, replace) Frame 3 [Page D, ref=1] When scanning: ref=0 → EVICT ref=1 → set bit=0, advance hand

Second Chance – Solved Trace

Reference string: 6, 0, 1, 2, 0, 3, 0, 5, 2, 3, 0, 3, 0, 3, 2, 1, 2, 0, 1, 6, 0, 1 | Frames = 3

Second Chance (Clock) Trace — F = Page Fault

Frame/Ref	6	0	1	2	0	3	0	5	2	3	0	3	0	3	2	1	2	0	1	6	0	1
Frame 1	6	6	6	2	2	2	2	5	5	3	3	3	3	3	3	3	3	0	0	0	0	0
Frame 2		0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1
Frame 3			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	2	2	6	6	6
Fault?	F	F	F	F		F		F	F	F						F		F		F		

Total Page Faults: 11 **Hits: 11** Hit Ratio: 50.0%

Second Chance Step-by-Step Walkthrough

Clock hand starts at Frame 0. Reference bits shown as [page, bit]:

Refs 6,0,1: All misses (empty frames). Frames: [6,0|ref=0], [0,0|ref=0], [1,0|ref=0]. Hand=0.

Ref 2: MISS. Hand=0, Frame0=[6,0] ref=0 → EVICT 6. Load 2. Hand→1. Frames: [2],[0],[1].

Ref 0: HIT on Frame1. Set Frame1 ref bit = 1. Frames: [2,0],[0,1],[1,0].

Ref 3: MISS. Hand=1 → Frame1=[0,ref=1] → give 2nd chance: set bit=0, hand→2. Frame2=[1,ref=0] → EVICT 1. Load 3. Hand→0. Frames: [2],[0],[3].

Ref 0: HIT on Frame1. Set ref=1. Frames: [2,0],[0,1],[3,0].

Ref 5: MISS. Hand=0 → Frame0=[2,ref=0] → EVICT 2. Load 5. Hand→1. Frames: [5],[0],[3].

Ref 2: MISS. Hand=1 → Frame1=[0,ref=1] → 2nd chance: bit=0, hand→2. Frame2=[3,ref=0] → EVICT 3. Load 2. Frames: [5],[0],[2].

Ref 3: MISS. Hand=0 → Frame0=[5,ref=0] → EVICT 5. Load 3. Frames: [3],[0],[2].

Refs 0,3,0,3,2: All HITS. Reference bits updated accordingly.

Ref 1: MISS. Clock scans; evicts least recently given 2nd chance frame.

Remaining refs: similar pattern of hits and replacement.

Algorithm	Page Faults (3 frames, this reference string)
FIFO	15 faults
Second Chance	11 faults
LRU	14 faults (on OPT's reference string)
OPT	9 faults (on OPT's reference string — minimum possible)

□ Exam Tip: Second Chance can achieve the same or better performance than FIFO because pages with a reference bit = 1 get a reprieve. It approximates LRU cheaply.

12. Frame Allocation – Concepts & Numericals

Frame allocation determines how many physical frames each process receives and which frames to replace on a fault.

Minimum Frames Per Process

Every process must have a minimum number of frames to execute even one instruction. If a single instruction can reference multiple pages, all those pages must be in memory simultaneously.

Example – IBM 370 MOVE instruction needs 6 frames: 2 for the instruction itself + 2 for source operand pages + 2 for destination operand pages.

Fixed Allocation Schemes

Scheme	Formula & Example
Equal Allocation	Each process gets: $\text{Total Frames} / \text{Number of Processes}$ Example: 100 frames, 5 processes → 20 frames each
Proportional Allocation	Each process gets: $(\text{process_size} / \text{total_size}) \times \text{total_frames}$ Smaller processes get fewer frames; larger get more

Proportional Allocation – Worked Example

Total frames = 64, p1 size = 10 pages, p2 size = 127 pages

```
Total size = 10 + 127 = 137 pages
p1 frames = (10 / 137) × 64 = 4.67 ≈ 5 frames
p2 frames = (127 / 137) × 64 = 59.33 ≈ 59 frames
Total allocated = 5 + 59 = 64 ✓
```

Proportional Allocation – Lecture Example

Total frames = 100, 5 processes equal size:

```
Each process = 100 / 5 = 20 frames
```

If processes have sizes 10, 20, 30 (total=60), frames=100:

```
p1 = (10/60) × 100 = 16.7 ≈ 17 frames
p2 = (20/60) × 100 = 33.3 ≈ 33 frames
p3 = (30/60) × 100 = 50.0 = 50 frames
Total = 17+33+50 = 100 ✓
```

Priority Allocation

Frames are distributed by process priority rather than size. A high-priority process may take frames from lower-priority processes. If a high-priority process faults, it can steal a frame from a lower-priority one.

Global vs Local Replacement

Global Replacement	Local Replacement
Process can take ANY frame from any other process	Process can only replace its OWN frames
Better throughput (most common)	More consistent per-process performance
Process execution time can vary greatly	May underutilize memory if process has spare frames
A process's fault rate depends on OTHER processes' behaviour	Fault rate depends only on process's own access pattern

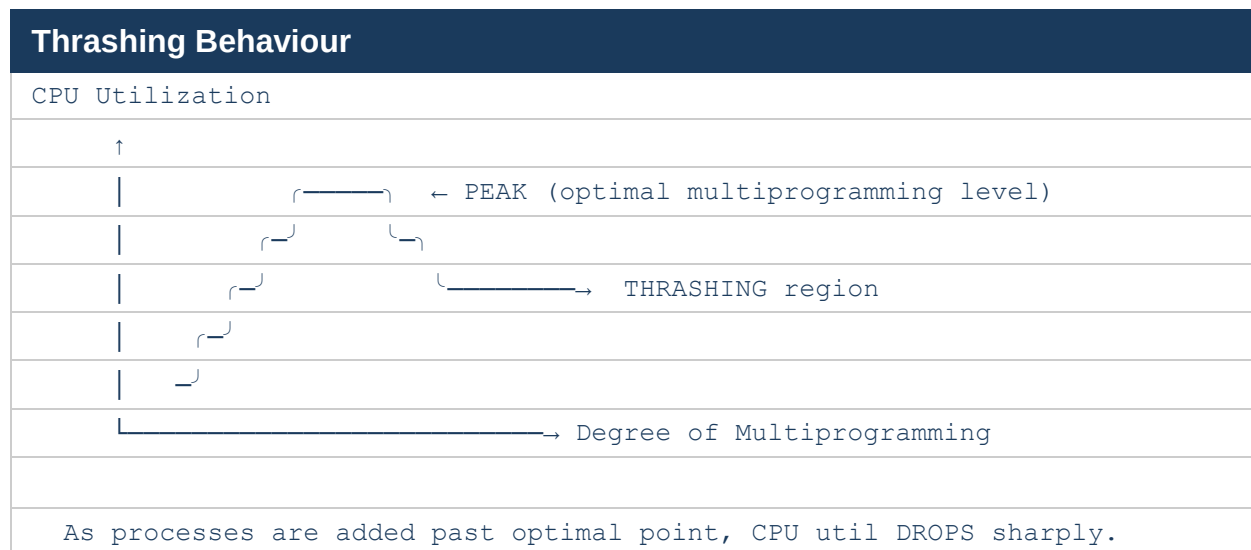
13. Thrashing

Thrashing occurs when a process does not have enough frames to hold its working set, causing an extremely high page-fault rate.

Thrashing Cycle

24. Process does not have enough frames → page fault rate very high.
25. Process faults → needs page → replaces existing frame → immediately needs replaced frame back.
26. OS sees low CPU utilization (all time spent on I/O waiting for pages).
27. OS increases degree of multiprogramming (adds another process).
28. New process steals frames → existing processes fault even more.
29. CPU utilization drops further → system grinds to a halt.

Thrashing Diagram (CPU Utilization vs Multiprogramming)



□ Working-Set Model: track the set of pages a process actively uses (its 'working set'). Give each process enough frames for its working set. If total working sets exceed physical memory → reduce multiprogramming.

14. Buddy System – Kernel Memory Allocation

The buddy system is used for allocating kernel (not user) memory. It allocates from a fixed-size physically contiguous segment using powers of 2.

Rules

- Memory is allocated in units that are powers of 2 (e.g., 1, 2, 4, 8, 16, 32 KB...).
- A request is rounded UP to the next power of 2.
- If a block of the right size is not available, a larger block is SPLIT into two equal buddies.
- Continue splitting until the right size is reached.
- When a block is freed, it is COALESCED (merged) with its buddy if the buddy is also free.

Worked Example (from lecture)

Available: 256 KB contiguous block. Request: 21 KB

```
Step 1: 21 KB → round up to next power of 2 = 32 KB
Step 2: Available = 256 KB. 256 > 32, so split:
        256 KB → AL = 128 KB + AR = 128 KB
Step 3: 128 KB > 32 KB, split AL:
        128 KB → BL = 64 KB + BR = 64 KB
Step 4: 64 KB > 32 KB, split BL:
        64 KB → CL = 32 KB + CR = 32 KB
Step 5: CL = 32 KB matches needed size → ALLOCATE CL to process

Internal Fragmentation = 32 KB - 21 KB = 11 KB wasted
```

Buddy System Tree	
[256 KB]	
/ \	
[128 KB]	[128 KB] ← AR (free)
/ \	
[64 KB]	[64 KB] ← BR (free)
/ \	
[32 KB]	[32 KB] ← CR (free)
↑	
Allocated (21 KB request, rounded to 32 KB)	

Coalescing (Merging Buddies)

When CL is freed:

30. Free CL (32 KB) → its buddy CR is also free → merge to BL (64 KB).
31. BL (64 KB) → buddy BR is free → merge to AL (128 KB).
32. AL (128 KB) → buddy AR is free → merge to original 256 KB block.

Buddy System Advantage	Buddy System Disadvantage
Fast coalescing — $O(\log n)$ to find buddy	Internal fragmentation — up to 50% of allocated block
Simple bookkeeping — buddies easily located	Memory wasted by rounding up to power of 2
Contiguous physical pages provided	Fragmentation gets worse as block size increases

Additional Buddy System Examples

Example 2: 512 KB pool, request = 70 KB

```
Round up 70 KB → 128 KB (next power of 2 after 70)
512 KB → 256 KB + 256 KB
256 KB → 128 KB + 128 KB
128 KB allocated. Internal fragmentation = 128 - 70 = 58 KB
```

Example 3: Power-of-2 rounding chart

Request Size	Allocated Block	Internal Frag
3 KB	4 KB	1 KB
21 KB	32 KB	11 KB
33 KB	64 KB	31 KB
65 KB	128 KB	63 KB
100 KB	128 KB	28 KB
200 KB	256 KB	56 KB

□ Exam Tip: Exam questions on Buddy System typically ask: (1) draw the splitting tree, (2) state the allocated block size, (3) calculate internal fragmentation = allocated - requested.

15. Slab Allocator

The slab allocator is an alternative kernel memory scheme designed to eliminate internal fragmentation for fixed-size kernel objects (e.g., process descriptors, file objects).

Key Terms

Term	Definition
Slab	One or more physically contiguous pages
Cache	Collection of one or more slabs; ONE cache per data structure type
Object	Instantiation of the kernel data structure stored in a slab

Operation

33. When a cache is created, it is filled with objects marked FREE.
34. When a structure is needed, a free object is marked USED and returned.
35. When the structure is released, the object is marked FREE again (kept in cache).
36. If a slab is full of used objects, a new empty slab is allocated.

Slab Allocator Benefit	Why It Helps
No fragmentation	Objects are exactly the right size for their type
Fast allocation	Pre-allocated objects ready to use immediately
Fast deallocation	Objects returned to free pool — no OS call needed
Cache-friendly	Objects don't move; memory locations reused efficiently

16. Other Considerations

Prepaging

Prepaging loads multiple pages into memory before they are needed to reduce the large number of page faults at process startup.

- Assume s pages prepaged; α fraction are actually used.
- Cost saved = $s \times \alpha$ page faults avoided.
- Cost wasted = $s \times (1 - \alpha)$ unnecessary pages loaded.
- If $\alpha \approx 0$ (most prepaged pages unused): prepaging HURTS. Only worthwhile if α is close to 1.

I/O Interlock (Page Pinning)

Some pages MUST be locked (pinned) in memory and cannot be swapped out:

- Pages used for device I/O (DMA buffers) must stay at fixed physical addresses.
- Pinned pages are excluded from the page replacement algorithm's victim pool.
- The kernel sets a lock/pin bit in the page table entry.

17. Practice Questions – Theory & Reasoning

Q1. What is virtual memory? How is it different from physical memory?

Ans: Virtual memory is a technique that separates logical (user) address space from physical RAM.

Physical memory = actual RAM chips; Virtual memory = abstraction of a larger address space backed by disk swap space.

Key difference: processes think they have more memory than physically exists. The OS + hardware manage the illusion.

Virtual memory can be implemented via demand paging or demand segmentation.

Q2. What is demand paging? Why is it called 'lazy' loading?

Ans: Demand paging brings a page into memory ONLY when it is actually referenced (demanded).

It is 'lazy' because no page is loaded proactively — the swapper (pager) waits until a page is needed.

Benefits: less I/O, less memory usage, faster process start, more concurrent users.

Q3. Explain the steps taken by the OS when a page fault occurs.

Ans (6 steps):

1. CPU accesses address → MMU finds valid-invalid bit = i → TRAP to OS.
2. OS checks internal table: invalid reference → abort; just not in memory → continue.
3. OS finds a free frame.
4. OS schedules disk operation to swap the page into the free frame.
5. OS updates page table: set frame number + valid bit = v.
6. OS restarts the faulting instruction.

Q4. What is Belady's Anomaly? Which algorithms suffer from it?

Ans: Belady's Anomaly is the counter-intuitive phenomenon where adding MORE frames to a FIFO system can result in MORE page faults.

Example: 3 frames may produce fewer faults than 4 frames for certain reference strings under FIFO.

FIFO suffers from Belady's Anomaly. OPT and LRU do NOT (they are stack algorithms).

Q5. Compare FIFO, OPT, and LRU page replacement algorithms.

FIFO: Replace oldest page. Simple, no hardware needed. Suffers Belady's anomaly. Worst performance.

OPT: Replace page used furthest in future. Minimum faults possible. NOT implementable (requires future knowledge). Used as benchmark.

LRU: Replace least recently used page. Near-optimal. Does NOT suffer Belady's. Needs hardware (counters/stack). Expensive.

Q6. What is the Second Chance (Clock) algorithm? How does it improve on FIFO?

Ans: Second Chance is FIFO + a reference bit per page. Pages with ref bit = 1 get a 'second chance' (bit cleared, page stays) before eviction.

Clock hand sweeps frames in FIFO order: ref=0 → evict; ref=1 → clear bit, advance hand.

Improvement over FIFO: recently-used pages avoid eviction, approximating LRU behaviour without needing counters or stacks.

Q7. Explain Equal and Proportional frame allocation with examples.

Equal: Divide total frames equally among all processes. 100 frames, 5 processes → 20 each. Simple but ignores process sizes.

Proportional: Allocate frames based on process size. formula: $\text{frames}_i = (\text{size}_i / \text{total_size}) \times \text{total_frames}$.

Example: p1=10 pages, p2=127 pages, 64 total frames → p1 gets 5, p2 gets 59.

Q8. What is thrashing? What causes it and how is it prevented?

Ans: Thrashing is when a process's page fault rate is so high that the CPU spends more time swapping pages than executing instructions.

Cause: too many processes with too few frames each. Process needs page P → evicts P' → immediately needs P' back.

Prevention: Working-Set Model (give each process its working set of frames), Page Fault Frequency (monitor fault rate and add/remove frames dynamically), reduce multiprogramming.

Q9. Explain the Buddy System. What are its advantages and disadvantages?

Ans: Buddy system allocates kernel memory in power-of-2 sized blocks. Request is rounded up to next power of 2. If the right size is unavailable, a larger block is split in half (creating 'buddies') until the correct size is reached.

Advantage: Fast coalescing — when a block is freed, if its buddy is also free, they merge instantly.

Disadvantage: Internal fragmentation (up to ~50%) because requests are rounded up to powers of 2.

Q10. Differentiate Global and Local page replacement.

Global: Process selects victim from ALL frames in the system (can steal from other processes). Higher throughput, but process fault rate depends on other processes. More common in practice.

Local: Process can only replace its OWN frames. More predictable per-process performance but may underutilize memory.

18. Practice Questions – Numericals (with Solutions)

Q1. EAT: $m_a = 150$ ns, page fault service time = 10 ms, $p = 1/500$. Calculate EAT.

Ans: Convert 10 ms = 10,000,000 ns

$$\text{EAT} = (1 - 0.002) \times 150 + 0.002 \times 10,000,000$$

$$= 0.998 \times 150 + 20,000$$

$$= 149.7 + 20,000 = 20,149.7 \text{ ns} \approx 20.15 \mu\text{s}$$

Slowdown = $20,149.7 / 150 \approx 134\times$ compared to no page faults.

Q2. $m_a = 100$ ns, clean page fault time = 10 ms, dirty page fault time = 20 ms, 60% of faults involve dirty pages, $p = 0.005$. Find EAT.

Ans: Weighted fault time = $0.60 \times 20,000,000 + 0.40 \times 10,000,000$

$$= 12,000,000 + 4,000,000 = 16,000,000 \text{ ns}$$

$$\text{EAT} = (1 - 0.005) \times 100 + 0.005 \times 16,000,000$$

$$= 0.995 \times 100 + 80,000$$

$$= 99.5 + 80,000 = 80,099.5 \text{ ns} \approx 80.1 \mu\text{s}$$

Q3. $m_a = 200$ ns, page fault time = 8 ms. What is the maximum page fault rate p that allows $\text{EAT} \leq 600$ ns?

Ans: $600 \geq (1-p) \times 200 + p \times 8,000,000$

$$600 \geq 200 - 200p + 8,000,000p$$

$$400 \geq 7,999,800p$$

$$p \leq 400 / 7,999,800 \approx 0.00005 \text{ (1 fault per 20,000 accesses)}$$

Q4. Run FIFO on: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 with 3 frames. Count page faults.

Ans: Step-by-step (Queue = oldest→newest):

1: Miss. {1} Queue=[1]

2: Miss. {1,2} Queue=[1,2]

3: Miss. {1,2,3} Queue=[1,2,3]

4: Miss. Evict 1. {4,2,3} Queue=[2,3,4]

1: Miss. Evict 2. {4,1,3} Queue=[3,4,1]

2: Miss. Evict 3. {4,1,2} Queue=[4,1,2]

5: Miss. Evict 4. {5,1,2} Queue=[1,2,5]

1: HIT

2: HIT

3: Miss. Evict 1. {5,3,2} Queue=[2,5,3]

4: Miss. Evict 2. {5,3,4} Queue=[5,3,4]

5: HIT

Total page faults = 9

Q5. Run OPT on: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3 with 3 frames. Count page faults.

Ans:

7: Miss. {7}

0: Miss. {7,0}

1: Miss. {7,0,1} — frames full

2: Miss. Next uses: 7=never, 0=pos5, 1=never → evict 7 (or 1, both not used again; pick 7). {2,0,1}

0: HIT

3: Miss. Next uses: 2=pos9, 0=pos7, 1=never → evict 1. {2,0,3}

0: HIT

4: Miss. Next uses: 2=pos9, 0=never again, 3=pos10 → evict 0. {2,4,3}

2: HIT

3: HIT

Total page faults = 6

Q6. Buddy System: A 1 MB (1024 KB) pool receives request for 100 KB. Show splits and internal fragmentation.

Ans: Round up 100 KB to next power of 2 = 128 KB

1024 KB → 512 KB + 512 KB

512 KB → 256 KB + 256 KB

256 KB → 128 KB + 128 KB

128 KB allocated.

Internal fragmentation = 128 – 100 = 28 KB wasted.

Three splits performed. Remaining free: 512 + 256 + 128 = 896 KB.

Q7. Total frames = 180. Process sizes: A=20 pages, B=50 pages, C=30 pages. Find frames allocated under proportional allocation.

Ans: Total pages = 20 + 50 + 30 = 100

A: $(20/100) \times 180 = 36$ frames

B: $(50/100) \times 180 = 90$ frames

C: $(30/100) \times 180 = 54$ frames

Total = 36 + 90 + 54 = 180 ✓

Q8. Run Second Chance on: 1, 2, 3, 2, 1, 5, 2, 1, 3 with 3 frames. Count page faults.

Ans: Frames=[], ref bits=[], hand=0

1: Miss. Frames=[1,_,_] bits=[0,0,0] hand→1. Faults=1

2: Miss. Frames=[1,2,_] bits=[0,0,0] hand→2. Faults=2

3: Miss. Frames=[1,2,3] bits=[0,0,0] hand→0. Faults=3

2: HIT. Set Frame[1] ref=1. bits=[0,1,0]

1: HIT. Set Frame[0] ref=1. bits=[1,1,0]
 5: Miss. Scan from hand=0: Frame[0]=[1,ref=1]→clear bit,hand→1.
 Frame[1]=[2,ref=1]→clear bit,hand→2. Frame[2]=[3,ref=0]→EVICT 3.
 Frames=[1,2,5] bits=[0,0,0] hand→0. Faults=4
 2: HIT. bits=[0,1,0]
 1: HIT. bits=[1,1,0]
 3: Miss. Scan hand=0: [1,ref=1]→clear,hand→1. [2,ref=1]→clear,hand→2.
 [5,ref=0]→EVICT 5. Frames=[1,2,3] bits=[0,0,0] hand→0. Faults=5
 Total page faults = 5

Q9. Run LRU on: 1, 2, 3, 4, 2, 1, 5, 3, 2, 1 with 3 frames. Count page faults.

Ans: (LRU order = most recent on right)

1: Miss. Frames={1} LRU=[1]. Faults=1
 2: Miss. Frames={1,2} LRU=[1,2]. Faults=2
 3: Miss. Frames={1,2,3} LRU=[1,2,3]. Faults=3
 4: Miss. LRU=1→evict. Frames={4,2,3} LRU=[2,3,4]. Faults=4
 2: HIT. LRU=[3,4,2]
 1: Miss. LRU=3→evict. Frames={4,2,1} LRU=[4,2,1]. Faults=5
 5: Miss. LRU=4→evict. Frames={5,2,1} LRU=[2,1,5]. Faults=6
 3: Miss. LRU=2→evict. Frames={5,3,1} LRU=[1,5,3]. Faults=7
 2: Miss. LRU=1→evict. Frames={5,3,2} LRU=[5,3,2]. Faults=8
 1: Miss. LRU=5→evict. Frames={1,3,2} LRU=[3,2,1]. Faults=9
 Total page faults = 9

Q10. A buddy system has 64 KB. After allocating 10 KB, 12 KB is freed in the sequence. Trace the splits and merges.

Ans: Round 10 KB → 16 KB (next power of 2).

64 KB → 32 KB + 32 KB

32 KB → 16 KB + 16 KB

Allocate 16 KB. Internal frag = 16 – 10 = 6 KB.

State: [16 KB USED] [16 KB free] [32 KB free]

When freed: 16 KB block freed → buddy (other 16 KB) is free → merge to 32 KB.

32 KB → buddy (other 32 KB) is free → merge to 64 KB original block.

19. Quick Reference Summary

Formula / Concept	Expression
EAT (Virtual Memory)	$(1-p) \times ma + p \times \text{page_fault_time}$
EAT (with TLB, Ch9)	$\alpha \times ma + (1-\alpha) \times 2 \times ma$ (TLB access ≈ 0)
Equal Allocation	$\text{Total_Frames} / \text{Num_Processes}$
Proportional Allocation	$(\text{size}_i / \text{total_size}) \times \text{total_frames}$
Buddy Block Size	Next power of 2 $\geq$ request size
Buddy Internal Frag	Allocated block – actual request
Page Fault Overhead	fault overhead + swap out (if dirty) + swap in

Algorithm	Replace	Faults (example)
FIFO	Oldest page in memory	15 (22-ref string, 3 frames)
Second Chance	Oldest with ref=0 (or give 2nd chance)	11 (same string)
LRU	Least recently USED	14 (23-ref string, 3 frames)
OPT	Furthest future use	9 (same string) – MINIMUM possible

Key Term	One-Line Definition
Virtual Memory	OS technique: logical space larger than physical RAM, backed by disk
Demand Paging	Load page only when accessed (lazy loading)
Page Fault	Access to a page not in physical memory $\rightarrow$ OS trap
Valid-Invalid Bit	v=page in memory; i=page on disk (triggers fault)
Lazy Swapper / Pager	OS component that swaps pages on demand
Belady's Anomaly	FIFO paradox: more frames $\rightarrow$ more faults for some strings
Thrashing	Too many faults $\rightarrow$ CPU spends time swapping, not executing
Working Set	Set of pages a process actively uses in recent Δ accesses
COW	Copy-on-Write: share pages after fork; copy only on write
Buddy System	Kernel memory: powers-of-2 allocation with buddy merging

Slab Allocator	Fixed-size object caches for kernel data structures
Global Replacement	Victim can be any frame (any process)
Local Replacement	Victim only from faulting process's own frames
Pinning / I/O Lock	Lock a page in memory; excluded from replacement
Prepaging	Load pages before they are referenced to reduce startup faults